Evidencia de que el problema es real y relevante

Por ejemplo en nuestra aula, las fallas se conocen cuando alguien avisa. Por ejemplo, en programación, si durante una práctica o un examen una computadora pierde la conexión o se cae la red, el docente se entera recién cuando el estudiante levanta la mano, y no queda registro de cuándo empezó el problema ni cuánto duró.

Con el sistema de monitoreo se ve en tiempo real qué equipos están en línea y queda un historial con fecha y hora.

Y el problema es real, y lo respaldan dos cosas. La primera es el ejemplo: apagamos una máquina virtual y comparamos cuánto tarda una persona en notarlo frente a lo que tarda el sistema en detectarlo. La segunda es que existen herramientas profesionales como Zabbix o Nagios, creadas precisamente porque las organizaciones tienen esta necesidad.

Cómo comprobaremos que el proyecto tuvo éxito

Definimos el éxito con resultados medibles, que se verifican en una demostración final en vivo:

1. El sistema detecta si un dispositivo está en línea o fuera de línea en **menos de 1 minuto**.
2. Se genera una **alerta** para el administrador en ese momento.
3. El dashboard muestra el estado de **al menos 5 dispositivos** y se actualiza solo, sin recargar la página.
4. Queda un **historial** con la fecha y hora en que cada dispositivo se apagó y volvió a encenderse.
5. Todo funciona sobre el servidor Proxmox del laboratorio.

La prueba final consiste en apagar una máquina virtual, ver cómo cambia a “fuera de línea”, recibir la alerta, volver a encenderla y comprobar que se registra la recuperación.

Criterios de aceptación por funcionalidad

Funcionalidad	Criterio verificable
Registro de dispositivos	Se puede crear, editar y eliminar un equipo; no acepta IPs duplicadas ni formatos inválidos
Monitoreo periódico	Cada dispositivo se revisa en el intervalo configurado
Detección de caída	Al apagar una VM pasa a “fuera de línea” en menos de 1 minuto
Latencia	El valor mostrado coincide con el que da el comando ping ejecutado a mano
Alertas	Por cada caída se genera una alerta visible para el administrador

Dashboard	Muestra el estado actual de todos los dispositivos y se actualiza solo
Histórico	Cada caída y recuperación queda guardada con fecha y hora y se puede consultar
Autenticación	Sin iniciar sesión no se puede ver ni modificar nada

Cada criterio se comprueba con una acción concreta: apagar una VM, comparar con ping, intentar entrar sin cuenta.

18. Funcionalidades interesantes pero innecesarias

Identificamos ideas atractivas que no son necesarias para el objetivo de detectar y registrar fallas: una aplicación móvil, notificaciones por WhatsApp o Telegram, integración con cámaras.

Las dejamos fuera de esta versión para no poner en riesgo el cronograma, y quedan documentadas como mejoras futuras. Las alertas dentro del sistema ya cumplen la función de avisar al administrador.

20. Qué conservaríamos y qué eliminaríamos si hubiera que reducir el proyecto 40%

Conservaríamos el núcleo: registro de dispositivos, monitoreo ICMP, detección de caídas y recuperaciones, histórico, dashboard con el estado actual, alerta básica y autenticación.

Eliminaríamos el SNMP y el monitoreo de tráfico, los reportes avanzados (dejando solo un resumen simple de disponibilidad) y los extras del dashboard, como los gráficos elaborados.

Así el sistema sigue resolviendo el problema central: saber qué dispositivo cayó, cuándo, y poder demostrarlo.

> Si te preguntan el cálculo del 40%, responde que es aproximado: SNMP, reportes y extras del dashboard equivalen a unas 3 o 4 de las 11 actividades del Gantt.

Validación de tecnologías críticas, especialmente SNMP

La integración más riesgosa es SNMP, por cuatro razones:

- A diferencia del ping, depende de que cada dispositivo lo tenga **habilitado y configurado**, y no todos lo traen activo.
- Requiere que la red permita su puerto (**UDP 161**).
- Hay que conocer los identificadores de cada dato (**OID**), que cambian según el fabricante.
- El tráfico no se obtiene directamente: los contadores se leen dos veces y se calcula la diferencia.

Además, es una tecnología que todavía no hemos implementado, por eso la validamos temprano:

1. Habilitamos SNMP en una máquina virtual de prueba.
2. Comprobamos desde fuera, con herramientas estándar (snmpwalk y snmpget), que responde con datos reales: nombre del equipo, tiempo encendido y contadores de tráfico.
3. Probamos la lectura desde C# contra esa misma máquina.

Si SNMP falla o no se logra a tiempo, el sistema sigue funcionando con ICMP. SNMP es una mejora, no un requisito para que el sistema funcione.

29. Dependencias externas, permisos y accesos

1. **Servidor Proxmox** encendido, donde corren todas las máquinas virtuales.
2. **Acceso a la red o VPN**, para que los tres integrantes lleguen al servidor.
3. **Direcciones IP fijas**, para que los dispositivos no cambien de IP.
4. **ICMP permitido** en la red y los firewalls; si no, todos aparecerían "fuera de línea".
5. **SNMP habilitado** en los equipos, para leer tráfico e interfaces.
6. **Permisos de administrador (sudo)** en las VMs, para instalar servicios y abrir puertos.
7. **GitHub**, para respaldo y control de cambios del código.
8. **Imágenes ISO** de Ubuntu Server y Alpine, para crear o reconstruir las VMs.
9. **SDK de .NET y sus paquetes**, para desarrollar y compilar.

Las dependencias que más cuidamos son el servidor Proxmox y la red, porque si fallan nadie puede trabajar.

Restricciones de seguridad, privacidad, permisos y acceso

- **Autenticación obligatoria:** nadie ve ni modifica nada sin iniciar sesión.
- **Contraseñas con hash**, nunca en texto plano.
- **Roles básicos:** administrador (registra, edita y elimina) y lector (solo consulta).
- **Datos guardados:** nombre, IP, estado y mediciones técnicas de los equipos. No se guardan datos personales de usuarios ni el contenido de lo que navegan.
- **SNMP:** usamos solo lectura, es decir, el sistema puede consultar datos de los equipos pero no cambiar su configuración. Cambiamos la comunidad por defecto (public), porque la conoce cualquiera, por una propia. Si el entorno lo permite, usamos SNMPv3, que agrega usuario, contraseña y cifrado.
- **Mínimos privilegios:** el servicio de monitoreo corre con un usuario sin permisos de administrador, y la base de datos no se expone a la red.
- **Aislamiento:** el panel de Proxmox y la base de datos no se publican en Internet; se accede por red interna o VPN.

Justificar que las estimaciones de tiempo del Gantt son razonables

El cronograma se considera razonable porque distribuye el trabajo en etapas y asigna las tareas según las responsabilidades y los conocimientos de cada integrante. La presentación estima 240 horas de trabajo, distribuidas de la siguiente manera:

Fase	Horas estimadas	Justificación
Análisis y requisitos	30 h	Definir el problema, los requisitos y el alcance del MVP.
Diseño	35 h	Diseñar la arquitectura, la base de datos y la estructura de la interfaz.
Desarrollo	100 h	Implementar el monitoreo ICMP, el dashboard, el historial, las alertas y las funcionalidades complementarias que alcancen el cronograma.
Pruebas	50 h	Verificar la detección de caídas, la recuperación, las mediciones, la autenticación y la integración de los componentes.
Documentación	25 h	Preparar el manual, actualizar la documentación y organizar la demostración final.
Total	240 h	

Las estimaciones son iniciales, no garantías exactas. Se consideran razonables porque el proyecto se construye por etapas, utiliza tecnologías conocidas como C# y .NET, y dispone de un entorno de pruebas basado en Proxmox y máquinas virtuales.

Además, se reserva tiempo para las pruebas y las correcciones, ya que integrar el monitoreo, la base de datos y el dashboard puede revelar problemas que no se detectan al desarrollar cada componente por separado.

Para comprobar que las estimaciones se cumplen, registraremos las horas reales dedicadas a cada tarea, compararemos el avance con el Gantt y reajustaremos las actividades si aparecen retrasos. Si el tiempo resulta insuficiente, priorizaremos el MVP y dejaremos las funcionalidades complementarias para futuras versiones.

38. Identificar explícitamente las dependencias entre tareas

Las tareas tienen dependencias porque algunas necesitan que otras hayan terminado, al menos parcialmente, para poder avanzar correctamente.

1. Análisis y levantamiento de requisitos

Define qué se construirá y qué se considerará terminado.

2. Diseño de arquitectura y base de datos

Depende de los requisitos principales y establece cómo se comunicarán los componentes.

3. Entorno de pruebas y núcleo de monitoreo ICMP

El entorno debe estar disponible y la lógica de monitoreo debe producir estados y mediciones utilizables.

4. Integración del dashboard, historial y alertas

Necesita datos del monitoreo y una estructura acordada para almacenarlos y consultarlos.

5. Pruebas integrales, correcciones y demostración

Dependen de que los componentes principales estén integrados y disponibles en el entorno de prueba.

También existen dependencias específicas:

- Entorno de pruebas: debe estar accesible para probar el envío de pings, las caídas de dispositivos y las recuperaciones.
- Base de datos: su estructura debe acordarse antes de integrar la persistencia del historial con el servicio de monitoreo y la interfaz.
- Dashboard: necesita una fuente estable de estados y mediciones para representar información correcta.
- Alertas e historial: necesitan que el sistema detecte los cambios de estado y determine cuándo un dispositivo cae o se recupera.
- SNMP y tráfico: dependen de la disponibilidad de dispositivos compatibles, la configuración del protocolo y la validación de las lecturas.
- Pruebas finales: necesitan una versión integrada y suficientemente estable del sistema.

Estas dependencias se deben documentar en el repositorio y revisar durante la reunión semanal. El diagrama de Gantt muestra el orden temporal de las actividades, pero conviene complementarlo con estas relaciones explícitas.

40. Identificar qué tareas podrían bloquear el trabajo del resto del equipo

Las principales tareas que podrían bloquear a otros integrantes son las siguientes:

Tarea crítica	Responsable principal	A quién puede bloquear y por qué
Preparar Proxmox, las máquinas virtuales y la conectividad	Josías	Puede impedir que el equipo pruebe el monitoreo y valide la integración.
Definir la arquitectura y el modelo de datos	Edilson y Luis	Puede bloquear la conexión entre el servicio, la base de datos y el dashboard si no se acuerdan las estructuras.
Implementar el núcleo ICMP	Josías	Puede bloquear el historial, las alertas y la visualización de estados reales.
Acordar las interfaces de comunicación y los datos que entrega el servicio	Josías y Luis, con coordinación de Edilson	Puede impedir que la interfaz consuma correctamente los resultados del monitoreo.
Integrar y probar el sistema completo	Los tres integrantes	Puede retrasar la corrección de errores, la documentación y la demostración final.
Validar SNMP	Josías	Puede retrasar el monitoreo de tráfico si se considera obligatorio; por eso no

		debe bloquear el MVP basado en ICMP.

Para reducir estos bloqueos, acordaremos desde el inicio el formato de los datos, la estructura de la base de datos, las interfaces entre componentes y las tareas que deben estar terminadas antes de comenzar otras.

Además, trabajaremos con entregas pequeñas y frecuentes en GitHub. Si una tarea crítica se retrasa, el responsable lo comunicará en la reunión semanal y se buscará apoyo de otro integrante.

La regla principal será no hacer depender el MVP de una funcionalidad complementaria. En particular, SNMP no debe impedir la entrega de la detección de caídas, el historial, el dashboard y las alertas básicas.

Definir qué hacer si un integrante deja de estar disponible

Si un integrante deja de estar disponible temporalmente o abandona el proyecto, el equipo aplicará un procedimiento de continuidad para evitar que su ausencia detenga el desarrollo.

1. Identificar las tareas afectadas. Se revisará qué actividades tiene asignadas el integrante y cuáles dependen de su trabajo.
2. Consultar la documentación y el repositorio. El equipo utilizará GitHub, los comentarios del código, las instrucciones de instalación y el registro de tareas para conocer el estado del trabajo.
3. Redistribuir las tareas prioritarias. Los integrantes disponibles asumirán primero las actividades necesarias para terminar el MVP.
4. Asignar un responsable sustituto. Cada tarea crítica tendrá un integrante que pueda continuarla si el responsable principal no está disponible.
5. Reducir el alcance si fuera necesario. Se aplazarán SNMP, los reportes avanzados y los elementos visuales secundarios antes de eliminar funciones esenciales.
6. Actualizar el cronograma. Se revisarán las fechas y responsabilidades, y se informará al docente si el cambio afecta significativamente la planificación.

Para prevenir este riesgo, mantendremos el código en GitHub, documentaremos las decisiones técnicas y procuraremos que al menos otro integrante conozca los aspectos esenciales de cada componente. También tendremos reuniones semanales para comunicar los avances y los problemas encontrados.

Distribución de respaldo propuesta:

Área principal	Responsable principal	Respaldo propuesto
Análisis, planificación y documentación	Edilson	Luis
Lógica de monitoreo e integración ICMP/SNMP	Josías	Edilson, con apoyo de Luis para la integración
Interfaz, dashboard y datos	Luis	Josías para la integración de datos, con apoyo de Edilson en la coordinación

Esta distribución es una propuesta de continuidad, no una afirmación de que todos ya dominen las tareas de los demás. Para hacerla efectiva, cada integrante deberá explicar y documentar su trabajo para que otro pueda retomarlo.

Confirmar que no quedan decisiones fundamentales pendientes que bloqueen el desarrollo

Las decisiones principales del proyecto ya están definidas: el problema que se busca resolver, los usuarios objetivo, el alcance del MVP, la tecnología general, el entorno de pruebas y los criterios de éxito. Por tanto, el desarrollo del núcleo del sistema puede comenzar sin esperar a resolver todas las funcionalidades complementarias.

Sin embargo, antes de integrar los componentes, debemos cerrar algunos detalles técnicos y organizativos.

Decisión o definición	Estado propuesto	Acción necesaria
Objetivo y alcance del MVP	Definido	Mantener el alcance acordado durante el desarrollo.
Tecnología principal: C# y .NET 10	Definida en la presentación	Confirmar el entorno de desarrollo y compilación de todos los integrantes.

Monitoreo mediante ICMP	Definido	Acordar el intervalo de comprobación, los tiempos de espera y cómo confirmar una caída.
Detección en menos de un minuto	Criterio de éxito definido	Ajustar el intervalo y los reintentos, y verificarlo con pruebas reales.
Base de datos y estructura de las mediciones	Parcialmente definida	Acordar las entidades, los campos y las relaciones antes de la integración.
Comunicación entre el servicio y el dashboard	Requiere detalle técnico	Definir el mecanismo de consulta o actualización de los estados.
Autenticación y roles	Requisitos definidos	Concretar el método de autenticación y aplicar los permisos de administrador y lector.
SNMP	Fuera del camino crítico del MVP	Validarlo por separado y no retrasar ICMP si la prueba falla.
Acceso a Proxmox, red y máquinas virtuales	Dependencia identificada	Verificar el acceso de los tres integrantes y los permisos necesarios.

En conclusión, no identificamos una decisión pendiente que deba bloquear el inicio del MVP. Sí hay detalles que deben resolverse antes de la integración. Para mantener el avance, registraremos cada decisión en la documentación y en GitHub, con un responsable y una fecha objetivo.

Aclarar los requisitos que todavía no se comprenden suficientemente, especialmente SNMP

El equipo distingue entre requisitos comprendidos y aspectos que necesitan una validación técnica. El objetivo no es resolver todos los detalles antes de empezar, sino identificar qué se debe comprobar y cómo.

Requisitos que debemos concretar

- Monitoreo ICMP: definir el intervalo entre comprobaciones, el tiempo de espera y la cantidad de intentos fallidos necesarios para marcar un dispositivo como fuera de línea. Estos parámetros deben permitir detectar caídas en menos de un minuto sin clasificar como caído un dispositivo por una pérdida aislada de paquetes.
- Latencia y pérdida de paquetes: definir cómo se calculan y muestran las mediciones y qué se hace cuando no hay respuesta.
- Historial: acordar qué eventos se almacenan, incluyendo caída, recuperación, fecha y hora, y qué ocurre cuando el servicio de monitoreo se reinicia.
- Actualización del dashboard: establecer cómo se actualizan los estados sin recargar la página.
- Alertas: definir cuándo se genera una alerta, cómo se identifica una recuperación y cómo evitar alertas duplicadas por una misma caída.
- Autenticación y roles: especificar qué operaciones puede realizar el administrador y cuáles están permitidas al lector.

Validación técnica de SNMP

SNMP es la parte con mayor incertidumbre técnica porque depende de la configuración y compatibilidad de los dispositivos. El plan de validación será el siguiente:

1. Preparar una máquina virtual de prueba que permita habilitar SNMP.
2. Verificar la conectividad y el acceso al puerto UDP 161 desde el equipo que realizará las consultas.
3. Usar snmpwalk y snmpget para comprobar que se pueden obtener datos reales.
4. Identificar los OID necesarios para consultar el nombre del dispositivo, su tiempo de actividad y los contadores de las interfaces.
5. Implementar una prueba en C# que lea esos valores.
6. Realizar dos lecturas de los contadores de tráfico y calcular la diferencia teniendo en cuenta el tiempo transcurrido y las unidades correspondientes.
7. Registrar los resultados, los problemas encontrados y los dispositivos compatibles.

Se utilizará acceso de solo lectura y se evitarán credenciales predeterminadas. Cuando sea posible, se preferirá SNMPv3 con autenticación y cifrado.

Criterio para decidir: si la consulta de SNMP funciona en el entorno de prueba y puede integrarse dentro del tiempo disponible, se incorporará al alcance complementario. Si no, se documentará el impedimento y se continuará con el MVP basado en ICMP. La disponibilidad, el estado de conexión, el historial y las alertas básicas no deben depender de SNMP.

Definir las condiciones que podrían provocar el fracaso del proyecto

El proyecto podría fracasar si no logra cumplir sus funciones principales dentro del plazo establecido, si el sistema no detecta las fallas de forma confiable o si los componentes no pueden integrarse correctamente.

Identificamos los siguientes riesgos y las medidas para reducirlos:

Riesgo de fracaso	**Consecuencia**	**Medida preventiva o correctiva**
Problemas de acceso al servidor Proxmox o a la red	El equipo no puede desarrollar o probar el sistema en el entorno previsto.	Verificar el acceso de los tres integrantes desde el inicio y preparar una máquina virtual alternativa para las pruebas locales.
ICMP bloqueado por un firewall o por la red	Los dispositivos pueden aparecer como fuera de línea aunque estén funcionando.	Verificar la conectividad y las reglas de firewall antes de probar el monitoreo; documentar las limitaciones de la red.
Detección de caídas superior a un minuto	No se cumple uno de los principales criterios de éxito.	Ajustar los intervalos y tiempos de espera, y comprobarlos apagando y encendiendo máquinas virtuales.
Errores en el historial o en las alertas	No se puede demostrar cuándo ocurrió una interrupción ni cuándo se recuperó un dispositivo.	Probar los cambios de estado y sus registros, incluidos los reinicios del servicio.
Problemas de integración entre monitoreo, base de datos y dashboard	Los componentes funcionan por separado, pero el sistema completo no.	Definir anticipadamente las interfaces, integrar de manera incremental y realizar pruebas frecuentes.

SNMP no funciona en el entorno disponible	No se pueden consultar los datos de tráfico previstos.	Validarlo temprano y mantenerlo como funcionalidad complementaria, sin bloquear el MVP basado en ICMP.
Ausencia de un integrante	Se retrasan tareas que dependen de una sola persona.	Documentar el trabajo, asignar respaldos y redistribuir las actividades prioritarias.
Estimaciones insuficientes o acumulación de errores	No se llega a terminar antes de la fecha de entrega.	Revisar el Gantt semanalmente, probar desde etapas tempranas y reducir las funcionalidades complementarias.
Pérdida de código o cambios incompatibles	Se pierde trabajo o dejan de funcionar componentes que ya estaban implementados.	Usar GitHub, realizar commits frecuentes y probar los cambios antes de integrarlos.
Falta de pruebas o documentación	No se puede demostrar correctamente el funcionamiento ni explicar cómo ejecutar el sistema.	Reservar tiempo específico para pruebas, documentación y ensayo de la demostración final.

¿Cuándo consideraríamos que el proyecto está en riesgo de fracasar?

Establecemos estas señales de alerta:

- El entorno de pruebas sigue sin estar disponible cuando debe comenzar la validación del núcleo ICMP.
- El sistema no consigue detectar de forma confiable una caída en menos de un minuto.
- El historial, las alertas o el dashboard siguen sin integrarse al acercarse la fase de pruebas.
- Se acumulan retrasos que ponen en peligro la fecha de entrega del 25 de noviembre de 2026.
- Una funcionalidad complementaria, como SNMP, consume tiempo que debería dedicarse a terminar el MVP.

- La ausencia de un integrante deja tareas críticas sin un responsable capaz de continuarlas.

Si aparece alguna de estas señales, el equipo revisará las prioridades y aplicará un plan de recuperación: resolver primero los bloqueos, redistribuir tareas y reducir el alcance complementario. Se conservarán las funcionalidades esenciales: registro de dispositivos, monitoreo ICMP, detección de caídas y recuperaciones, historial, dashboard, alertas básicas y autenticación.

Criterio final: consideraremos que el proyecto ha fracasado en su objetivo principal si no podemos demostrar, en el entorno del laboratorio, que el sistema detecta las caídas, informa del cambio de estado y registra las interrupciones y recuperaciones de manera confiable. Si alguna mejora complementaria no se completa, se documentará como trabajo futuro, sin confundirlo con el fracaso del MVP.